

1. Checklist de Teste Estático de Código (Revisão Manual)

Projeto: Simulador de Batalha - PokeSal / **Componente:** Análise Estática de Código

Avaliador: João Gabriel Moreira Carvalho Araujo

Resumo do Checklist

Item	Pergunta de Verificação	Conformidade	Resumo dos Achados
1.1	Todas as variáveis estão inicializadas antes do uso?	CONFORME	Atributos e variáveis locais inicializados corretamente.
1.2	Há variáveis declaradas e nunca usadas?	NÃO CONFORME	Identificadas variáveis mortas na classe <code>BatalhaService.java</code> .
1.3	Existe código inacessível (<i>Unreachable Code</i>)?	CONFORME	O fluxo estruturado impede blocos inalcançáveis.
1.4	Há código duplicado?	NÃO CONFORME	Duplicação pontual de lógica de sorteio de Enums e verificação.
1.5	Existem erros de sintaxe?	CONFORME	Código compila sem erros sintáticos após correção de OCR.
1.6	Há erros de lógica que quebram regras do negócio?	NÃO CONFORME	Desconformidades nas regras de Terreno e na validação do Antidote.

1.7	Há erros de tipagem?	NÃO CONFORME	Truncamento e casting implícito de <code>double</code> para <code>int</code> causam perda de precisão no cálculo de dano.
1.8	O fluxo de controle é válido (sem loops infinitos, condições impossíveis)?	CONFORME	Estruturas <code>while</code> e <code>do-while</code> possuem critérios claros de parada.
1.9	Outros tipos de erros ou inconsistências identificados?	NÃO CONFORME	Violação de encapsulamento no Enum <code>Itens</code> e padrão <code>Checkstyle</code> .

Detalhamento e Fundamentação Técnica

1.1. Todas as variáveis estão inicializadas antes do uso?

- **Status: CONFORME**
- **Análise:** Todos os atributos de classe do projeto possuem inicializações explícitas via construtor (ex.: `this.hpAtual = this.hpMax;` na classe `Pokesal`) ou nas próprias declarações de constantes. As variáveis locais declaradas em métodos como `BatalhaService.calcularDano()` e `Main.iniciarBatalha()` são devidamente atribuídas antes de qualquer operação de leitura ou retorno.

1.2. Há variáveis declaradas e nunca usadas?

- **Status: NÃO CONFORME**
- **Análise:**
 - Na classe `BatalhaService.java`, no método `atacar(Pokesal atacante, Pokesal defensor)`:
 - Java

`int danoSofrido = defensor.receberDano(dano);`

-
- A variável `danoSofrido` recebe o retorno de `defensor.receberDano(dano)`, mas é utilizada apenas na mensagem de log.
- Em `BatalhaService.java`, no cálculo do dano base do terreno, a classe do tipo `Terreno` define atributos e getters que acabam não sendo plenamente aproveitados nas comparações estáticas da batalha.

1.3. Existe código inacessível (*Unreachable Code*)?

- **Status: CONFORME**
- **Análise:** A análise estática de fluxo de controle confirmou a ausência de trechos inacessíveis. Todos os comandos após cláusulas `return` ou estruturas `break`

encontram-se fora de blocos inatingíveis, garantindo que todo o código compilado seja executável sob as condições adequadas.

1.4. Há código duplicado?

- **Status: NÃO CONFORME**
- **Análise:**
 - **Sorteio de Enum:** Nas classes `BatalhaService.java` e `Main.java`, a lógica para selecionar um valor aleatório de um Enum (`Terreno.values()`, `SalDex.values()`, `Itens.values()`) é duplicada em múltiplos métodos (`sortearEscolhaCpu`, `sortearItemRecompensa`, etc.).
 - **Processamento de Fim de Turno:** O método `processarFimDeTurno(pokesal)` é chamado de forma sequencial repetida dentro de `iniciarBatalha()` para o jogador e para a CPU, contendo validações idênticas de encerramento da batalha.

1.5. Existem erros de sintaxe?

- **Status: CONFORME**
- **Análise:** O código-fonte em Java apresenta sintaxe válida segundo o padrão JDK e compila normalmente sem erros de compilação. (Nota: Erros leves de digitação observados em tokens e separadores derivaram do processo de digitalização/OCR da documentação original e foram ajustados).

1.6. Há erros de lógica que quebram regras do negócio?

- **Status: NÃO CONFORME**
- **Análise das Regras de Negócio:**
 1. **Regra do Terreno Planta (Canteiro Central):** O requisito formal estabelece que Pokesals do tipo Planta recuperam 5% do HP máximo ao final de cada turno. Contudo, no método `calcularDano()` da `BatalhaService.java`, há a instrução:
 2. Java

```
if (terrenoAtual != null && atacante.getTipoElemental() != TipoElemental.PLANTA)
```

- 3.
4. Essa condição bloqueia o bônus de dano/efeito em terrenos para Pokésals de Planta de forma divergente da regra de negócio descrita no documento de especificação.
5. **Validação do Antidote (Mochila):** Em `Mochila.java`, o método `validarUsoltem()` exige que o Pokesal esteja obrigatoriamente com o status `Status.ENVENENADO` para usar o Antidote. No entanto, a mecânica da batalha permite aplicar o status de queimado ou paralisado sem um item correspondente de cura específica na regra atual.

1.7. Há erros de tipagem?

- **Status: NÃO CONFORME**
- **Análise:**

- No método `calcularDano()` de `BatalhaService.java`, operações matemáticas envolvendo porcentagens e multiplicadores utilizam o tipo `double` (`danoCalculado *= multiElemental;`), mas sofrem *casting* explícito direto para `int`:
- Java

```
int danoFinal = (int) danoCalculado;
```

-
- Isso causa **perda por truncamento** dos decimais em vez do arredondamento matemático correto, o que pode gerar discrepâncias sutis no cálculo de dano final e nos testes de *Boundary Values* (Valores Limite).

1.8. O fluxo de controle é válido (sem loops infinitos, condições impossíveis)?

- **Status: CONFORME**
- **Análise:** Os laços `while (!pokesalJogador.estaDerrotado() && !pokesalCpu.estaDerrotado())` na classe `BatalhaService` e o laço `do-while` na classe `Main` possuem critérios claros de parada. A leitura de opções via `Scanner` trata adequadamente entradas inválidas utilizando `scanner.next()` para descarte de buffers incorretos, prevenindo laços infinitos na entrada de dados via console.

1.9. Outros tipos de erros ou inconsistências identificados?

- **Status: NÃO CONFORME**
- **Análise:**
 - **Construtor do Enum `Itens`:** O construtor do Enum `Itens` apresenta corpo vazio ou incorretamente fechado:
 - Java

```
Itens(String nome, int valorCura, Status statusCura) {
}
this.nome = nome; // Linhas fora do bloco do construtor
```

-
- Isso fere a integridade sintática e de inicialização dos objetos do Enum.
- **Nomenclatura (Checkstyle / Google Java Style):** Algumas variáveis em `BatalhaService.java` usam espaços de formatação inconsistentes no nome e violações do padrão de convenção `camelCase`.